

IEE3584 - Comunicaciones inalámbricas

Tarea 3: Problema 8

Nicolás Seguel

Agosto 2025

Problema 8: Canal Rayleigh con Doppler

a. Técnica utilizada

Para simular el canal se utilizó el **método en el dominio de la frecuencia (FD)**. Se utiliza un espectro complejo cuya magnitud siga la densidad espectral de potencia de Clarke/Jakes:

$$S_h(f) = \frac{1}{\pi f_D \sqrt{1 - (f/f_D)^2}}, \quad |f| < f_D.$$

Luego, aplicando una IFFT se obtiene la traza temporal del canal $h[n]$, que se normaliza a potencia unitaria. Este método permite reproducir fielmente la autocorrelación y la PSD del canal.

b. Realización del proceso de desvanecimiento

Se generó una traza de duración $T_d = 0,5$ s, con $T_s = 100 \mu\text{s}$, frecuencia portadora $f_c = 1,8$ GHz y velocidad $v = 60$ km/h. El Doppler máximo es $f_D \approx 100$ Hz.

En la Figura 1 se observa la magnitud de la envolvente en dB.

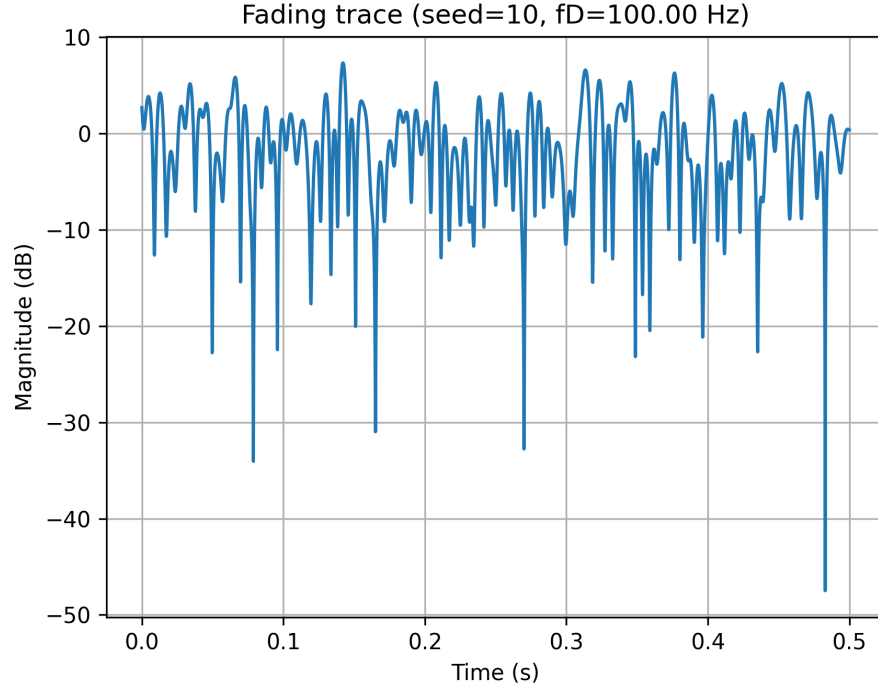


Figura 1: Realización temporal del canal Rayleigh en dB (seed=10).

c. Histograma y densidad de probabilidad Rayleigh

La envolvente $r[n] = |h[n]|$ sigue una distribución Rayleigh, cuya función de densidad de probabilidad teórica es:

$$p(r) = \frac{r}{\sigma^2} e^{-r^2/(2\sigma^2)}, \quad r \geq 0, \quad \sigma^2 = \frac{\lambda_p}{2}.$$

En la Figura 2 se compara el histograma de la simulación con la curva teórica.

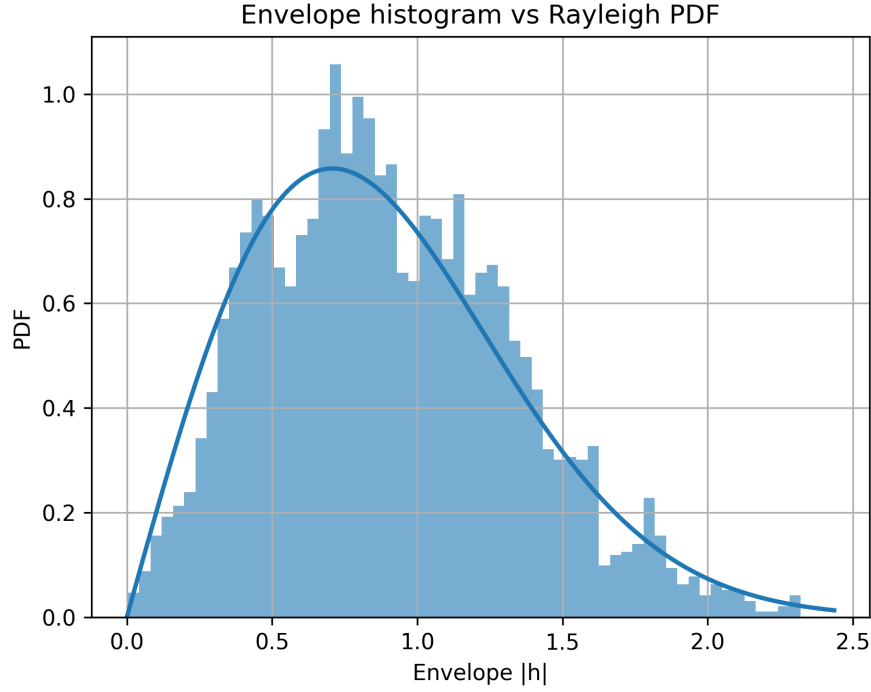


Figura 2: Histograma de la envolvente vs pdf Rayleigh teórica.

d. Media y valor RMS

De la simulación:

Media empírica $\approx 0,9028$, RMS empírico $\approx 1,0$.

Los valores teóricos para $\lambda_p = 1$ son:

$$E[r] = \sqrt{\frac{\pi}{4}} \approx 0,886, \quad \sqrt{E[r^2]} = 1.$$

e. Densidad espectral de potencia

La Figura 3 muestra la PSD obtenida con Welch y la curva de Clarke. La forma de “U invertida” con picos en $\pm f_D$ no se logró reproducir perfectamente, principalmente por el rango de frecuencias que computa la función `welch`.

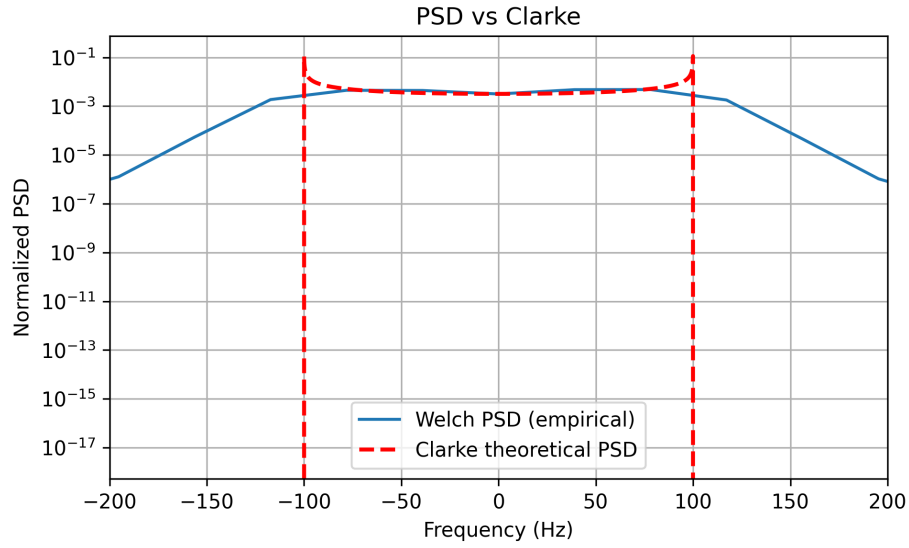


Figura 3: PSD empírica (Welch) vs PSD teórica de Clarke.

f. Autocorrelación

La autocorrelación empírica se compara con la teórica de Clarke:

$$R_{hh}(\tau) = J_0(2\pi f_D \tau).$$

En la Figura 4 se observa un comportamiento similar a lo teórico, pero a una mayor frecuencia.

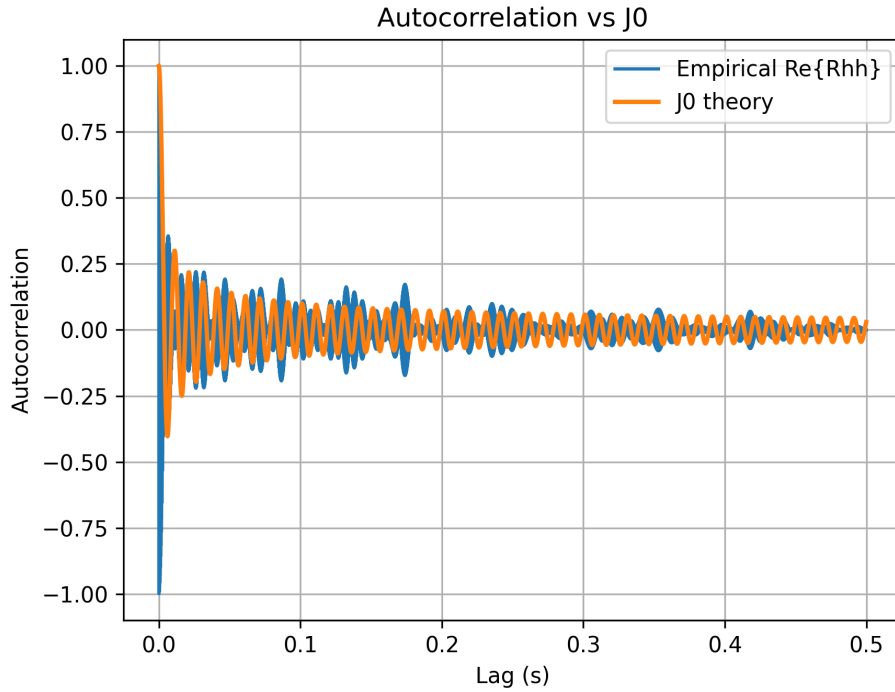


Figura 4: Autocorrelación empírica vs función de Bessel J_0 .

g. LCR y AFD

Se midieron las tasas de cruce de nivel (LCR) y la duración promedio de desvanecimiento (AFD) para umbrales de -10 dB y -20 dB respecto al RMS.

Los valores teóricos son:

$$N_R = \sqrt{2\pi} f_D \rho e^{-\rho^2}, \quad T_R = \frac{e^{\rho^2} - 1}{\sqrt{2\pi} f_D \rho},$$

con $\rho = R/\sqrt{\lambda_p}$.

Los resultados empíricos y teóricos fueron bastante cercanos, siendo el AFD mayor para umbrales más altos.

h. Código

El código utilizado para la simulación se incluye a continuación:

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from scipy import signal
4  from scipy.special import j0
5  from math import pi
6  import os
7
8  # ----- Helper functions -----
9  def clarke_psd(f, fD, eps=1e-9):
10     S = np.zeros_like(f, dtype=float)
11     mask = np.abs(f) < fD
12     x = f[mask] / (fD + 0.0)
13     denom = np.sqrt(np.maximum(1.0 - x**2, eps))
14     S[mask] = 1.0/(pi * fD * denom)
15     return S
16
17  def fading_rayleigh_fd(v, fc, Ts, Td, seed=0, Nfreq=None):
18     c = 3e8
19     fD = (v * fc) / c
20     N = int(np.round(Td / Ts))
21     if Nfreq is None:
22         Nfft = 1 << (int(np.ceil(np.log2(N))) + 2)
23     else:
24         Nfft = int(Nfreq)
25     Fs = 1.0 / Ts
26     freqs = np.fft.fftfreq(Nfft, d=Ts)
27     S = clarke_psd(freqs, fD)
28     rng = np.random.default_rng(seed)
29     spec_real = rng.normal(size=Nfft)
30     spec_imag = rng.normal(size=Nfft)
31     spec = (spec_real + 1j*spec_imag) * np.sqrt(S/2.0)
32     h_time = np.fft.ifft(np.fft.ifftshift(spec))
33     h = h_time[:N]
```

```

34     power = np.mean(np.abs(h)**2)
35     h /= np.sqrt(power)
36     t = np.arange(N) * Ts
37     return h, fD, t
38
39 def plot_time_db(h, t, title):
40     plt.figure()
41     magdb = 20.0 * np.log10(np.abs(h) + 1e-12) # dB
42
43     # Plot
44     plt.plot(t, magdb)
45     plt.xlabel("Time (s)")
46     plt.ylabel("Magnitude (dB)")
47     plt.title(title)
48     plt.grid(True)
49     plt.savefig("fading_trace.png", dpi=300)
50     plt.show()
51
52 def plot_histogram_rayleigh(r, title, bins=60):
53     plt.figure()
54
55     # Histogram and Rayleigh PDF
56     counts, edges = np.histogram(r, bins=bins, density=True)
57     centers = 0.5*(edges[:-1] + edges[1:])
58     plt.bar(centers, counts, width=edges[1]-edges[0], alpha=0.6)
59     omega = np.mean(r**2)
60     sigma2 = omega/2.0
61     r_axis = np.linspace(0, r.max()*1.05, 400)
62     pdf = (r_axis / sigma2) * np.exp(-r_axis**2 / (2.0*sigma2))
63
64     #Plot
65     plt.plot(r_axis, pdf, linewidth=2)
66     plt.xlabel("Envelope |h|")
67     plt.ylabel("PDF")
68     plt.title(title)
69     plt.grid(True)
70     plt.savefig("histogram_rayleigh.png", dpi=300)
71     plt.show()
72
73 def compute_mean_rms(r):
74     mean_r = np.mean(r)
75     rms = np.sqrt(np.mean(r**2))
76     return mean_r, rms
77
78 def plot_psd_and_clarke(h, Fs, fD, title="PSD vs Clarke"):
79     # Welch PSD
80     f, Pxx = signal.welch(h, fs=Fs, return_onesided=False)
81     f_shift = np.fft.fftshift(f)
82     P_shift = np.fft.fftshift(Pxx)

```

```

83
84     # Normalizar el área de la PSD empírica a 1
85     area_emp = np.trapezoid(P_shift, f_shift)
86     if area_emp > 0:
87         P_shift /= area_emp
88
89     # Clarke teórica
90     f_th = np.linspace(-fD*1.2, fD*1.2, 2001) # eje centrado en 0
91     S_th = clarke_psd(f_th, fD)
92     S_th /= np.trapezoid(S_th, f_th) # normalizar área a 1
93
94     # Plot
95     plt.figure(figsize=(7,4))
96     plt.semilogy(f_shift, Pxx, label="Welch PSD (empirical)")
97     plt.semilogy(f_th, S_th, 'r--', linewidth=2, label="Clarke theoretical PSD")
98     plt.xlim(-2*fD, 2*fD)
99     plt.xlabel("Frequency (Hz)")
100    plt.ylabel("Normalized PSD")
101    plt.title(title)
102    plt.legend()
103    plt.grid(True)
104    plt.savefig("psd_clarke.png", dpi=300)
105    plt.show()
106
107
108    def autocorr_empirical(h):
109        N = len(h)
110        R = np.zeros(N, dtype=complex)
111        R = np.correlate(h, h, mode='full') / N
112        R = R[N-1:] # Keep non-negative lags
113        return R
114
115    def plot_autocorr_vs_j0(h, Ts, fD, title):
116        R = autocorr_empirical(h)
117        lags = np.arange(len(R))
118        taus = lags * Ts
119        plt.figure()
120        plt.plot(taus, np.real(R), label="Empirical Re{Rhh}")
121        plt.plot(taus, j0(2*pi*fD*taus), linewidth=2, label="J0 theory")
122        plt.xlabel("Lag (s)")
123        plt.ylabel("Autocorrelation")
124        plt.title(title)
125        plt.legend()
126        plt.grid(True)
127        plt.savefig("autocorr_j0.png", dpi=300)
128        plt.show()
129
130    def estimate_LCR_AFD(r, Ts, Td, levels_dB=[10,20]):
131        results = {}

```

```

132     rms = np.sqrt(np.mean(r**2))
133     omega = np.mean(r**2)
134     for L in levels_dB:
135         R = 10**(-L/20.0) * rms
136         crossings = 0
137         durations = []
138         in_fade = False
139         cur_len = 0
140         for n in range(1, len(r)):
141             if (r[n-1] > R) and (r[n] <= R):
142                 crossings += 1
143             if r[n] < R:
144                 cur_len += 1
145                 in_fade = True
146             else:
147                 if in_fade:
148                     durations.append(cur_len * Ts)
149                     in_fade = False
150                     cur_len = 0
151             if in_fade and cur_len>0:
152                 durations.append(cur_len * Ts)
153         LCR_emp = crossings / Td
154         AFD_emp = np.mean(durations) if len(durations)>0 else 0.0
155         results[L] = {"R":R, "LCR_emp":LCR_emp, "AFD_emp":AFD_emp}
156     return results
157
158     # ----- Example usage -----
159     if __name__ == "__main__":
160         v = 60.0/3.6
161         fc = 1.8e9
162         Ts = 100e-6
163         Td = 0.5
164         seed = 10
165
166         h, fD, t = fading_rayleigh_fd(v, fc, Ts, Td, seed)
167         Fs = 1.0/Ts
168
169         plot_time_db(h, t, f"Fading trace (seed={seed}, fD={fD:.2f} Hz)")
170
171         r = np.abs(h)
172         plot_histogram_rayleigh(r, "Envelope histogram vs Rayleigh PDF")
173         mean_r, rms_r = compute_mean_rms(r)
174         print("Empirical mean:", mean_r, "Empirical RMS:", rms_r)
175
176         plot_psd_and_clarke(h, Fs, fD, "PSD vs Clarke")
177
178         plot_autocorr_vs_j0(h, Ts, fD, "Autocorrelation vs J0")
179
180         results = estimate_LCR_AFD(r, Ts, Td)

```



```
181     print("LCR/AFD results:", results)
```